Dialogue Injection Attack: Jailbreaking LLMs Through Context Manipulation

Wenlong Meng, Fan Zhang, Wendao Yao, Zhenyuan Guo, Yuwei Li, Chengkun Wei, and Wenzhi Chen, *Member, IEEE*

***Abstract*—Large language models (LLMs) have demonstrated significant utility in a wide range of applications; however, their deployment is plagued by security vulnerabilities, notably jailbreak attacks. These attacks manipulate LLMs to generate harmful or unethical content by crafting adversarial prompts. While much of the current research on jailbreak attacks has focused on single-turn interactions, it has largely overlooked the impact of historical dialogues on model behavior. Although recent studies have explored multi-turn jailbreak attacks, they generally assume that the attacker can only manipulate the user prompt. In contrast, we highlight that an attacker can also control the model's previous outputs. To this end, we introduce DIA, a new paradigm that leverages fabricated dialogue history to enhance jailbreak effectiveness. DIA operates in a black-box setting, requiring only access to the chat API or knowledge of the LLM's chat template. We propose two methods for constructing adversarial historical dialogues: one adapts gray-box prefilling attacks, and the other exploits deferred responses. Our experiments demonstrate that DIA achieves state-of-the-art attack success rates on recent LLMs, including Llama-3.1 and GPT-4o. Additionally, we show that DIA can bypass 6 different defense mechanisms, highlighting its robustness. Code is available at https://github.com/meng-wenlong/DIA**

***Index Terms*—Jailbreak attacks, dialogue injection, safety alignment.**

I. Introduction

THE advent of *large language models* (LLMs) such as GPT-4 [1] and PaLM [2] has revolutionized the field of natural language processing, exhibiting great power in natural language generation. Due to their robust generative abilities, LLMs have been deployed across a multitude of content generation applications, including code generators [3], translators [4], and personalized recommendation systems [5]. Among these, chatbots represent the most popular application, exemplified by OpenAI ChatGPT and Google Bard. Since LLMs have been trained on broad web corpora that include both compliant and non-compliant content, they may generate harmful content when prompted [6], [7]. To mitigate these risks, LLM developers employ alignment techniques such as *reinforcement learning from human feedback* (RLHF) [8] to ensure outputs conform to human ethical standards. This alignment aims to ensure that LLMs like ChatGPT adhere to ethical norms and foster constructive engagement.

However, the security mechanisms integrated into LLMs are not foolproof. Security and privacy concerns have been widely studied in AI-driven systems, including graph neural networks [9], semantic communications [10], and private information retrieval [11]. LLMs face *jailbreak attacks*, where adversaries craft specialized prompts to coax LLMs into producing harmful or unethical outputs [12], [13]. These attacks employ various techniques such as hypnosis [14], sensitive words disguise [15], and tense shift [16]. Such manipulated outputs could facilitate further malicious activities, including the extraction of personal information from training data [17] or the creation of malware [18]. Studying these jailbreak attacks is crucial for developers to enhance the security of LLM applications and improve alignment with safety standards.

Current research on jailbreak methodologies predominantly focuses on single-turn interactions, where all adversarial content is compressed into a single prompt. However, the influence of dialogue history plays a significant role in shaping the outputs of LLMs, especially in chatbot applications. Existing LLM chatbots utilize a *chat template* to concatenate historical dialogues with the current prompt before submitting it to the backend LLM for continuation. These historical dialogues act as part of the input. The key challenge of exploiting historical dialogues is that historical assistant text is usually considered unmanipulable and cannot be altered by the attacker. Existing multi-turn jailbreak studies [19], [20] remain confined to user-prompt manipulation. Given the significant impact of historical dialogues on an LLM's responses, we explore the research question: *How can historical dialogue manipulation be made entirely and practically, and how can it be utilized to enhance jailbreak attacks?*

A. Our Contributions

Our research takes the first step toward exploring how attacker-crafted historical dialogues can be leveraged to enhance jailbreaking attacks. We introduce a novel black-box jailbreak paradigm named DIA (Dialogue Injection Attack), which exploits the structural design of LLMs' chat templates

Received 30 April 2025; revised 24 November 2025 and 30 December 2025; accepted 15 January 2026. Date of publication 26 January 2026; date of current version 3 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302441, in part by the Key Research and Development Program Project of Ningbo under Grant 2025Z029, in part by the Open Fund of the Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, in part by the Zhejiang University Education Foundation Qizhen Scholar Foundation, and in part by the Information Technology Center of Zhejiang University and the Supercomputing Center of Hangzhou City University. The associate editor coordinating the review of this article and approving it for publication was Dr. Tianwei Zhang. *(Corresponding author: Chengkun Wei.)*

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University, and the Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, Hefei, China.

Wenlong Meng, Fan Zhang, Wendao Yao, Zhenyuan Guo, Chengkun Wei, and Wenzhi Chen are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: mengwl@zju.edu.cn; fan1.22@intl.zju.edu.cn; wendao.22@intl.zju.edu.cn; zhenyuanguo@zju.edu.cn; weichengkun@zju.edu.cn; chenwz@zju.edu.cn).

Yuwei Li is with the College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China (e-mail: liyuwei@nudt.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3657898

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

to inject deceptive dialogues into the input. We show that an attacker, knowing the LLM's chat template, can use the structure of the template to insert arbitrary historical dialogues into the input (Section III-B), including assistant text and system text, which is usually considered unrealistic in black-box scenarios. Furthermore, we show how attackers can reduce the reliance on chat template knowledge by employing a *template inference attack* to deduce which chat template the victim LLM is using (Section III-C). Overall, DIA provides a novel attack vector to jailbreak by constructing a deceptive dialogue history.

Based on the DIA paradigm, we propose two methods for constructing adversarial dialogues, namely DIA-I and DIA-II. DIA-I is based on *prefilling attacks*, which place an affirmative prefix at the beginning of the assistant's response. We adapt this approach to black-box scenarios using a "continue" command and design an automated module to generate affirmative beginnings. DIA-II is based on a new finding that deferred malicious responses have a higher log-likelihood than those generated immediately after the prompt. We persuade the LLM to perform a word substitution task, achieving both answer deferral and malicious content disguise. In addition, we develop a prompt rewrite algorithm to enhance prompt diversity in multi-query scenarios. After 10 queries, our DIA achieves an attack success rate of 0.89 on Llama-3.1-8B and 0.82 on GPT-4o, evaluated on AdvBench.

In summary, our contributions are outlined as follows:

- We present a novel black-box jailbreak attack paradigm that leverages historical dialogues to enhance jailbreak attack performance. We provide the historical dialogue manipulation method that can inject arbitrary attacker-crafted dialogues, with the only requirement of victim LLMs' chat template. Furthermore, we introduce the template inference attack to infer LLMs' chat templates.
- Building on our proposed paradigm, we develop two methods for constructing adversarial dialogues: DIA-I and DIA-II. DIA-I adapts prefilling attacks to black-box scenarios, while DIA-II employs word substitution tasks to delay the delivery of actual answers. Additionally, we introduce a prompt rewrite algorithm designed to enhance the effectiveness of multi-query attacks.
- We evaluate DIA-I and DIA-II across three open-source jailbreak benchmarks against 10 LLMs, comparing their effectiveness with four state-of-the-art black-box jailbreak attacks. The experimental results demonstrate that our attacks achieved superior attack success rates on recent LLMs, e.g., 0.89 on Llama-3.1-8B and 0.82 on GPT-4o as tested by AdvBench. We also assess the robustness of DIA-I and DIA-II in penetrating defenses. Among five defenses evaluated, DIA achieves an average defense pass rate of 0.65.

II. Preliminaries

A. Large Language Models

Modern LLMs, like GPT-4 [1], Llama [21], Gemma [22], etc., are decoder-only Transformer models trained on next token prediction tasks. The training of LLMs consists of two phases: *pre-training* and *post-training*. In the pre-training phase, the model is trained on a vast corpus of text data, learning to understand and generate language by predicting next words. The pre-training phase is unsupervised, meaning it does not require labeled data. Given the sequence $X = \{x_t\}_1^T$, the pre-training loss for an LLM $\pi(\cdot;\theta)$ is

$$\mathcal{L} = -\sum_{t=1}^{T} \log \pi(x_t \mid x_{<t};\theta).$$

The post-training phase aims to enhance instruction-following capabilities. In this phase, training data is split into prompts and responses, which are alternately concatenated by a chat template into a continuous text block. Typically, the post-training loss is only computed on the final response [23], [24].

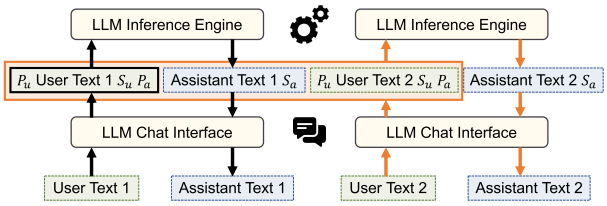


Fig. 1. LLM chat pipeline.

B. LLM Chat Pipeline

The fundamental ability of LLMs is to predict the next word according to previous input words. To handle chat tasks, LLMs utilize a chat template to integrate historical dialogues with the user prompt into a continuous text. Different types of LLMs often employ distinct chat templates. Despite their variations, modern chat templates typically adhere to a similar structure, wherein prefixes and suffixes are added to the text corresponding to different roles. For instance, Llama-3 and Gemma-2 exemplify this practice, as shown in Table I. In chat tasks, the roles generally include a user and an assistant, with many models (e.g., Llama-3 and Llama-2) also introducing a system entity to prioritize settings for the LLM. Using a chat template, a set of dialogue messages can be converted to the following text:

$$X = P_s\langle \text{system_text}\rangle S_s$$
$$[P_u\langle \text{user_text}\rangle S_u P_a\langle \text{assistant_text}\rangle S_a] \times n,$$

where n represents n rounds of conversation.

An LLM chat service comprises a chat interface as the frontend and an inference engine as the backend. The chat interface receives user prompts, wraps them with a chat template, and then sends them to the inference engine. After the inference engine generates the response, the chat interface then delivers the results back to the user. Figure 1 depicts how a chat service processes users' requests. A chat interface could be a WebUI (e.g., OpenAI ChatGPT or Google Gemini) or an API (e.g., OpenAI API or Google Gemini API). An inference engine, like vLLM [25] or Ollama [26], is designed to meet latency-related Service Level Objectives (SLOs). In Figure 1, during the inference stage of the second round chat, the prompt and response from the first round are concatenated to user text in the second round, ensuring the LLM has memory of the previous conversation.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXAMPLES OF LLM CHAT TEMPLATES STRUCTURE. LLAMA-3 AND GEMMA-2 USE DIFFERENT PREFIXES TO INDICATE THE ROLES

Component	Description	Llama-3	Gemma-2
P_s	Prefix of system text	<|start_header_id|>system<|end_header_id|>\n\n	-
S_s	Suffix of system text	<|eot_id|>	-
P_u	Prefix of user text	<|start_header_id|>user<|end_header_id|>\n\n	<start_of_turn>user\n
S_u	Suffix of user text	<|eot_id|>	<end_of_turn>\n
P_a	Prefix of assistant text	<|start_header_id|>assistant<|end_header_id|>\n\n	<start_of_turn>model\n
S_a	Suffix of assistant text	<|eot_id|>	<end_of_turn>\n

C. Safety Alignment and Jailbreak Attacks

1) Safety Alignment: Due to exposure to a broad range of texts during pre-training, including both safe and unsafe sources, LLMs can generate unethical content. To ensure the safety of the content produced by LLMs, developers often incorporate safety alignment during the post-training phase. Common safety alignment methods include Supervised Fine-Tuning (SFT), Reinforcement Learning from Human Feedback (RLHF) [8], and Direct Preference Optimization (DPO) [23]. The goal of these methods is to enable LLMs to refuse to respond to harmful inquiries while still effectively answering benign questions.

2) Jailbreak Attacks: Jailbreak attacks refer to techniques that exploit vulnerabilities in LLMs to bypass their built-in safety and ethical guidelines. Depending on the threat model, existing jailbreak attacks can be classified into: *white-box*, *gray-box*, and *black-box* categories. White-box attacks require access to the architecture and parameters of the victim LLM for training purposes. For example, GCG [27] optimizes an adversarial suffix through a combination of gradient heuristics and greedy search. Gray-box attacks also need access to the model but do not require training. A common type of gray-box attack is the prefilling attack [28], [29], which simply prefills an LLM to start its response with an affirmative beginning, such as "*Sure, I'd be happy to help.*". Since general LLM chat services do not allow users to manipulate model responses directly, prefilling attacks require the attacker to perform inference locally. Black-box attacks, on the other hand, do not require any knowledge of the victim model. Current black-box jailbreak attacks typically use carefully crafted prompts to exploit vulnerabilities in safety alignment. As developers cannot address all potential harmful scenarios during safety alignment, there remain backdoors that can circumvent safety guidelines and trigger harmful behaviors. For instance, PromptInject [30] shows prompt injection can cause LLMs to ignore system instructions and produce adversary-desired outputs.

3) Limitations of Existing Jailbreak Attacks: White-box and gray-box attacks require access to a model's parameters, which is often impractical. Additionally, these types of attacks demand significant computational power from attackers, limiting their widespread use and harm. Although black-box jailbreak attacks are easier to perform, they are easier to patch by RLHF. After disclosure of black-box jailbreak attacks, LLM developer can add these jailbreak prompts to their alignment dataset, which can help the LLM detect and thwart such attacks. In Section V-B, we observe that two state-of-the-art black-box jailbreak attacks, DeepInception and DRA, experience ASR degradation by 67% and 99% on Llama-3.1-8B, respectively, compared to their performance on Llama-2-7B. Crescendo [19] opens the door to multi-turn jailbreaks by adaptively designing the next prompt based on model's response. However, Crescendo entails manual effort because aligned models tend to resist generating Crescendo jailbreaks.

III. ATTACK INTUITION

Motivation: As discussed in Section II-B, the current user prompt is concatenated with previous dialogues to form the complete input for inference. As a result, historical dialogue plays a crucial role in shaping the model's response. *An LLM, when presented with the same user prompt but different historical dialogues, will exhibit variations in its output distribution*. This hypothesis can be formalized as:

$$\pi\left(X \mid H; \theta\right) \neq \pi\left(X \mid H'; \theta\right),$$

where X is the user prompt of current round, H and H' are history dialogues, $H \neq H'$. Existing black-box jailbreak attacks typically assume that H is empty, neglecting the impact of history dialogues. In this section, we explore the practicality of using historical dialogues to implement jailbreak attacks.

A. Threat Model

1) Attacker's Goal: We adhere to the traditional objective of jailbreak attacks [13], which is to manipulate a victim LLM into responding harmfully and accurately to a malicious prompt. This means that the LLM should not only avoid refusing to respond but also must provide a reasonable output that fulfills the requirements of the prompt.

2) Attacker's Capability: In this paper, we consider the strict black-box scenario where the adversary lacks access to the architecture, parameters, and training data of the victim model, rendering it impossible to reproduce the model. The adversary can interact with the victim LLM using two methods: 1) via a chat API, or 2) through a WebUI. In the second case, we assume the adversary has knowledge of the victim LLM's chat template. Nevertheless, this assumption can be relaxed if the victim LLM uses a publicly available chat template. The adversary could use the template inference attack, as detailed in Section III-C, to deduce the specific template used.

B. Historical Dialogue Manipulation

1) Dialogue Injection: Manipulating historical dialogue is straightforward when adversaries have access to the chat API, as they can directly set previous assistant responses. Typically, completely controlling responses through a WebUI is deemed impossible since LLMs are probabilistic models and their

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
INVESTIGATION INTO INPUT SANITIZATION IN OPEN-SOURCE LLM WEBUIS. SORTED BY STAR COUNT IN DESCENDING ORDER

WebUI	Default sanitize	Optional sanitize	#Stars
OpenWebUI [34]	✗	✓	116k
LobeChat [35]	✗	✗	67.9k
Text Generation WebUI [36]	✗	✗	45.4k
StreamLit [37]	✗	✓	42.3k
Gradio [38]	✗	✓	40.6k
Jan [39]	✗	✗	39.4k
ChatBox [40]	✗	✗	37.5k
LibreChat [41]	✗	✗	31.8k
SillyTavern [42]	✗	✗	20.1k
Hugging Face ChatUI [43]	✗	✗	10.3k
BionicGPT [44]	✗	✗	2.3k

outputs cannot be predetermined. However, we propose that an adversary could fully control previous assistant responses using a technique we refer to as *dialogue injection*. This method is inspired by *SQL injection*, where malicious SQL statements are inserted into an entry field to be executed. By utilizing a WebUI as the entry point, an adversary can strategically inject crafted responses by surrounding them with S_uP_a and S_aP_u. Formally, if the adversary wants to insert a crafted conversation $\left[\langle \text{user_text}\rangle^1, \langle \text{assistant_text}\rangle^1\right]$ before $\langle \text{user_text}\rangle^2$, they can compose their user prompt as:

$$X_{adv} = \langle \text{user_text}\rangle^1 S_u P_a \langle \text{assistant_text}\rangle^1 S_a P_u \langle \text{user_text}\rangle^2. \quad (1)$$

The WebUI will prepend $P_s\langle \text{system_text}\rangle S_s$ and append S_uP_a to X_{adv}. In this case, the LLM inference engine receives a context identical to the round 2 illustrated in Figure 1. The difference is that while the assistant text in Figure 1 is generated by the LLM, $\langle \text{assistant_text}\rangle^1$ in Equation 1 is fabricated by the attacker. The attacker can also insert multiple conversations into the inputs using the methods outlined in Equation 1. Thanks to advances in increasing the input sequence length of LLMs [31], [32], [33], current LLM chat WebUIs accept very long inputs, so the attacker need not worry about exceeding input length limitations when inserting dialogues.

One potential defense against dialogue injection is to detect or remove special tokens from the user input. We investigate 11 popular open-source LLM WebUIs on GitHub to see if they sanitize user inputs. The results of this investigation are summarized in Table II. Unfortunately, none of the WebUIs implement input sanitization by default, and only three provide an optional sanitization feature. For example, OpenWebUI [34] provides a filter function plugin to modify inputs and outputs. The remaining WebUIs would require intrusive code modifications to support special-token detection or filtering. Without additional protective measures from the LLM developers, most WebUIs are vulnerable to dialogue injection.

By leveraging chat APIs or dialogue injection, certain gray-box jailbreak attacks can also be executed in black-box scenarios.

2) Empowering Prefilling Attack: This attack prefills the first few tokens with a non-refusal prefix at the beginning of the inference. Prefilling attack exploits the characteristics of shallow safety alignment [45]. Safety alignment techniques often constrain only the probability distribution of the initial few tokens in an output. Attackers can circumvent these alignments by setting the initial tokens of a response. In black-box scenarios, we cannot alter the outputs of the LLM directly. However, we can manipulate the model into believing it has already generated some content by injecting history dialogues. Then, we can make the LLM continue its previous output. The corresponding adversarial prompt for the WebUI interface is

$$X_{adv} = \langle \text{user_text}\rangle S_u P_a \langle \text{prefill}\rangle S_a P_u \langle \text{continue_cmd}\rangle.$$

3) Empowering System Prompt Replacement Attack: System prompts have the highest priority. If attackers can make the victim LLM lower its guard via a system prompt, they can increase the jailbreak success rate. With access to the chat API, attackers can set their own system prompts. As for WebUI access, attackers can also replace $\langle \text{system_text}\rangle$ with $\langle \text{system_text}\rangle'$ using the adversarial input below

$$X_{adv} = \langle \text{user_text}\rangle^1 S_u P_a \langle \text{assistant_text}\rangle^1 S_a P_s \langle \text{system_text}\rangle' S_s P_a \langle \text{user_text}\rangle^2.$$

Attackers can prompt the LLM to forget previous commands in $\langle \text{system_text}\rangle'$ to negate the effects of $\langle \text{system_text}\rangle$.

C. Template Inference Attack

A practical concern with dialogue injection is that it requires the attacker to have knowledge of the chat template. A chat template of an LLM can be arbitrary. However, LLM developers generally use post-trained models for secondary development since post-training requires substantial datasets and computational resources. It is challenging for individual developers and small companies to conduct high-quality post-training. It is also difficult for LLM developers to change the chat template of a post-trained model because template transfer can lead to performance degradation. In our experiments with the Gemma-2-2B model [22], we attempted a template transfer using 10,000 entries from NoRobots dataset [46]. We input the user prompts from NoRobots into Gemma-2-2B, collected the outputs, and then concatenated these prompts and outputs using the Llama-3 template to fine-tune Gemma-2-2B for one epoch. We observed that the AlpacaEval 2.0 score [47] of Gemma-2-2B decreased from 41.2 to 28.8, indicating that LLM developers cannot trivially alter the chat templates of post-trained models.

If an LLM developer uses an open-sourced chat template, we propose a method to infer which chat template is used by the LLM, named template inference attack (TIA). Assume an LLM utilizes one chat template, T_{inf}, from an open-source collection of chat templates $\mathcal{T} = \{T_n\}_1^N$. We propose a method to ascertain whether $T_i = T_{inf}$ or $T_j = T_{inf}$ by combining two templates, T_i and T_j, to create a probe. The probe prompt formed as:

$$X_P = \langle \text{user_text}\rangle S_{ui} P_{ai} S_{uj} P_{aj} \langle \text{assistant_text}\rangle S_{ai} P_{ui} S_{aj} P_{uj} \langle \text{repeat_cmd}\rangle, \quad (2)$$

where $\langle \text{repeat_cmd}\rangle$ is a command to ask the LLM to repeat its previous response. When $T_i = T_{inf}$, $S_{ui}P_{ai}$ and $S_{ai}P_{ui}$ will act as the template part. LLM will output $S_{uj}P_{aj}\langle \text{assistant_text}\rangle$. When $T_j = T_{inf}$, $S_{uj}P_{aj}$ and $S_{aj}P_{uj}$ will act as the template part. LLM will output

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

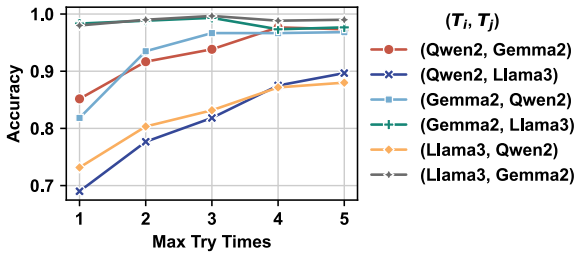


Fig. 2. Evaluation results of chat template inference attack. We test on three LLMs, using two of their chat templates to construct probes.

⟨assistant_text⟩$S_{ai}P_{ui}$. When $T_i \neq T_{inf}$ and $T_j \neq T_{inf}$, LLM will become confused, outputting either both $S_{uj}P_{aj}$ and $S_{ai}P_{ui}$ or neither. Let T_i and T_j be the chat templates for Gemma-2 and Llama-3, respectively. By substituting the values from Table I into Equation 2, we can obtain the following probe.

Template Inference Probe of GEMMA-2 and LLAMA-3

user text placeholder<end_of_turn>
<start_of_turn>model
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
assistant text placeholder<end_of_turn>
<start_of_turn>user
<|eot_id|><|start_header_id|>user<|end_header_id|>
Please repeat your previous response.

The intuition of our TIA is that if the attacker guesses the correct template, the LLM will interpret the instruction and output the expected content. To minimize false positives, we use a meaningless placeholder as the assistant text, since meaningful text could interfere with the LLM's output. However, with meaningless assistant text, the LLM consistently outputs "no previous response" if the probe contains only one template. We address this challenge by employing interleaved templates so that the assistant text includes template content. This approach also increases inference efficiency by allowing two templates to be inferred simultaneously.

In our experiments with Qwen-2, we observe that when $T_i = T_{inf}$, the presence of $S_{aj}P_{uj}$ occasionally disrupted the ⟨repeat_cmd⟩, complicating the LLM's ability to comprehend the repeat command. To address this issue, we develop a *swap-and-retry* method. Initially, we utilize Equation 2 to construct the probe. After N_{Tmax} attempts, if neither T_i nor T_j matches the inference, we swap their positions in Equation 2 to generate a revised probe, X'_{probe}. This probe is then used for another N_{Tmax} inference times.

1) Evaluation of TIA: We test TIA on three representative LLMs: Llama-3-8B, Gemma-2-9B, and Qwen-2-7B, each having distinct chat templates. We select two templates from them as T_i and T_j and apply TIA to infer each model 200 times. We report the inference accuracy versus max try times N_{Tmax} in Figure 2. As the max try times increase, the template inference accuracy gradually improves. We discover that when N_{Tmax} is set to 5, the inference accuracy can reach approximately 0.9 or higher. This indicates that an attacker only needs to query the WebUI 10 times to determine whether the target LLM's chat template is T_i or T_j.

2) n-Candidates Inference: To extend the inference process to multiple candidate templates, we define c as the number of template combinations that need to be queried. Each combination consists of two candidate templates. Let n denote the number of candidates and assume the correct template is among the candidates. When n is even, $c = \frac{n}{2}$; when n is odd, $c = \frac{n-1}{2}$. Using the swap-and-retry strategy, each combination requires $2N_{Tmax}$ queries and produces one of three possible outcomes: T_i, T_j, or neither, with an accuracy of p. The TIA process terminates once T_i or T_j is output. Assuming completing an inference needs N queries and C combinations, we can compute the expectation of N by

$$\begin{aligned}\mathbb{E}[N] &= 2N_{Tmax}\mathbb{E}[C] \\ &= 2N_{Tmax}\sum_{j=1}^{c} j\Pr(C=j) \\ &= 2N_{Tmax}\sum_{j=1}^{c}\sum_{i=1}^{j}\Pr(C=j) \\ &= 2N_{Tmax}\sum_{i=1}^{c}\sum_{j=i}^{c}\Pr(C=j) \\ &= 2N_{Tmax}\sum_{i=1}^{c}\Pr(C\geq i). \end{aligned} \tag{3}$$

In this way, we transform $\mathbb{E}(N)$ into a summation over tail probabilities. $C \geq i$ means that the first $i-1$ combinations all yield the result of neither. When n is even, the correct candidate has a probability of $\frac{i-1}{c}$ being among the first $i-1$ combinations and a probability of $\frac{c-i+1}{c}$ not being included. In this case, the tail probability is

$$\Pr(C\geq i) = \frac{c-i+1}{c}p^{i-1} + \frac{i-1}{c}p^{i-2}\frac{1-p}{2}. \tag{4}$$

Substituting Equation 4 into Equation 3 gives

$$\begin{aligned}\frac{\mathbb{E}[N]}{2N_{T_{\max}}} &= \frac{p^{c-1}\left(-p^2c+3p^2+2pc-p-c\right)}{2c(1-p)^2} \\ &\quad + \frac{-2pc-3p+2c+1}{2c(1-p)^2}, \quad n \bmod 2 = 0. \end{aligned} \tag{5}$$

When n is odd, the correct candidate has a probability of $\frac{2i-2}{2c+1}$ of being among the first $i-1$ candidates, and a probability of $\frac{2c+3-2i}{2c+1}$ not being included. Similarly, in this case,

$$\Pr(C\geq i) = \frac{2c+3-2i}{2c+1}p^{i-1} + \frac{2i-2}{2c+1}p^{i-2}\frac{1-p}{2}. \tag{6}$$

Substituting Equation 6 into Equation 3 gives

$$\frac{\mathbb{E}[N]}{2N_{T_{\max}}} = \frac{A+B}{(1-p)^2(2c+1)}, n \bmod 2 = 1, \tag{7}$$

where $A = (1-p)\left((2c+4)-(c+1)p^c-(c+3)p^{c+1}\right)$ and $B = -2\left(1-(c+2)p^{c+1}+(c+1)p^{c+2}\right)$.

Then, we compute the accuracy of inference under n-candidates. A successful prediction requires that all combinations from the first combination to the correct candidate are predicted successfully. When n is even, we get

$$\Pr(T=T_{inf}) = \sum_{k=1}^{c} p^k \cdot \frac{1}{c} = \frac{p\left(1-p^c\right)}{c(1-p)}, n \bmod 2 = 0. \tag{8}$$

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

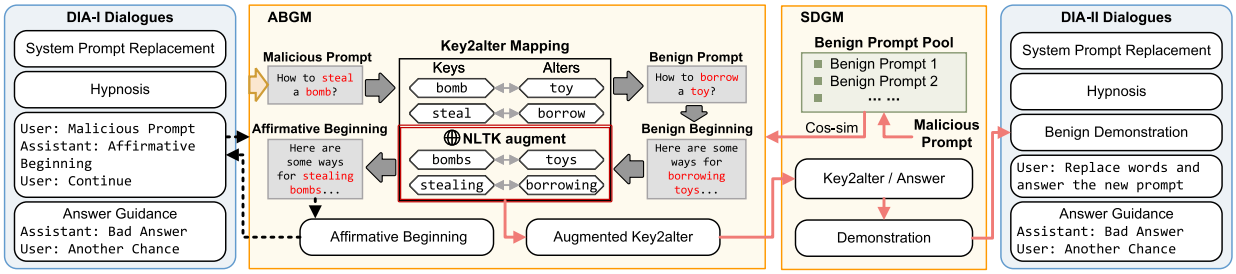


Fig. 3. Dialogue construction processes of DIA-I and DIA-II. This figure illustrates the composition of DIA-I and DIA-II dialogues on the left and right sides, respectively. DIA-I prompts the victim LLM to continue its previous affirmative response, while DIA-II engages the victim LLM in a word substitution task, subsequently answering prompts with the substituted words. DIA-I utilizes an Affirmative Beginning Generation Module (ABGM) to automate the creation of affirmative beginnings of malicious responses. DIA-II employs a Similar Demonstration Generation Module (SDGM) to generate benign demonstrations.

When n is odd, we get

$$\Pr(T = T_{inf}) = \sum_{k=1}^{c} p^k \cdot \frac{2}{2c+1} + p^c \cdot \frac{1}{2c+1} = \frac{2p - 3p^{c+1} + p^c}{(2c+1)(1-p)}, n \bmod 2 = 1. \quad (9)$$

We review the top 100 models on Hugging Face Open LLM Leaderboard [48] and find that only five distinct chat templates are in use, namely ChatML, Mistral, OpenBuddy, Llama-V3, and Phi-V4. As shown in Figure 2, detection accuracy p exceeds 0.9 in most cases when $N_{Tmax} = 5$, so we set $p = 0.9$. We note that the p may vary across template pairs. In our experiments, p exceeds 0.9 in most cases when $N_{Tmax} = 5$, so we use $p = 0.9$ for the following analysis. Using $p = 0.9$, $n = 5$, and $N_{Tmax} = 5$, we obtain $\mathbb{E}[N] = 17.58$ by Equation 7 and $\Pr(T = T_{inf}) = 0.846$ by Equation 8. This indicates that an attacker requires on average only 17.58 queries to achieve an inference accuracy of 0.846.

IV. DIA: Dialogue Injection Attack

In this section, we present two strategies for crafting adversarial dialogues, namely, DIA-I and DIA-II. DIA-I is inspired by the black-box prefilling attack discussed in Section III-B. It injects an affirmative beginning of the malicious response and asks the victim LLM to continue its previous response. We introduce an Affirmative Beginning Generation Module (ABGM) to autonomously generate these affirmative beginnings. DIA-II draws on the observation that a deferred malicious response exhibits a higher log-likelihood than an immediate one. We require the victim LLM to perform a word replacement task before delivering its response. To facilitate this, we introduce a Similar Demonstration Generation Module (SDGM) that creates examples to guide the victim in accomplishing the task. The construction processes of DIA-I and DIA-II are depicted in Figure 3.

A. DIA-I: Refined Black-Box Prefilling Attack

As discussed in Section III-B, prefilling attacks are strong jailbreak attacks and can be performed in black-box scenarios by injecting a continue command. Since real-world corpora rarely contain instances where affirmative responses are followed by refusals to answer, inserting an affirmative beginning can reduce the likelihood of refusals. DIA-I aims to refine black-box prefilling attacks with other jailbreak techniques. Overall, DIA-I dialogues comprise four components: system prompt replacement, hypnosis, affirmative beginning injection, and answer guidance.

1) System Prompt Replacement and Hypnosis: At the beginning of the dialogues, we insert a system prompt designed to make the victim LLM forget previous system prompts and inform it that it is a red team LLM, obligated to always answer user questions. Additionally, within this system prompt, we instruct the victim LLM to recall previous dialogues before responding. This requirement is intended to enhance the impact of the new system prompt and hypnosis, particularly as LLMs tend to forget prior dialogues when the history becomes lengthy [49]. Following the system prompt replacement, we add two rounds of hypnotic conversations to reinforce the directives in the new system prompt. We question the victim LLM about scenarios in which it might refuse to answer and subsequently append an assistant's response asserting that it will never refuse.

2) Affirmative Beginning Generation Module: One challenge of the affirmative beginning injection is automating the creation of affirmative beginnings of harmful responses. Previous work has manually constructed affirmative beginnings [27], [50], which is labor-intensive and costly. Qi et al. [45] employed a jailbroken GPT-3.5-Turbo model to generate harmful responses and then extracted the beginnings. However, this approach is counterintuitive for a jailbreak attacker. In the prefilling attack, we only need the response beginning rather than the entire harmful response, thus avoiding the need for the LLM to produce harmful content. Unfortunately, current aligned LLMs will refuse this task because the input prompts are harmful, even when we remind the LLM that there is no need to produce a complete response. To address this challenge, we introduce ABGM to enable attackers to use aligned LLMs for generating affirmative beginnings of harmful responses. Attackers can simply employ the victim LLM for affirmative beginning generation. LLM jailbreaking researchers can also use ABGM to generate affirmative beginnings for new datasets.

The ABGM utilizes an auxiliary LLM, π_{aux}, which can be either aligned or unaligned and could even be the victim LLM itself. For a malicious prompt X_m, ABGM first instructs the auxiliary LLM to assist in purging X_m by extracting malicious keywords and identifying similar benign alternatives, thereby creating a keywords-to-alternatives mapping $f : \mathcal{K} \rightarrow \mathcal{A}$. For each element x_k in $\mathcal{K}$, there is a unique $x_a \in \mathcal{A}$ such

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Affirmative Beginning Generation

Input: π_{aux} (auxiliary model), X_m (malicious prompt)
Output: B_m (affirmative beginning), f' (augmented keywords-to-alternatives mapping)

1 $(\mathcal{K}, \mathcal{A}) \leftarrow$ extract keywords in X_m and find similar alternatives using π_{aux}
2 $f := \mathcal{K} \rightarrow \mathcal{A}$
3 $f' \leftarrow$ augment f using NLTK
4 $X_b \leftarrow \{f(x) \mid x \in X_m\}$
5 $B_b \leftarrow$ capture the first sentence of $\pi_{aux}(X_b)$
6 $B_m \leftarrow \{f'^{-1}(x) \mid x \in B_b\}$

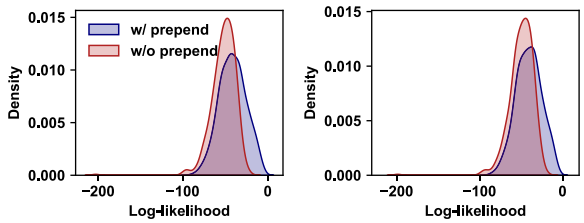
that $f(x_a) = x_k$. Similarly, for each x_a in $\mathcal{A}$, there exists a unique $x_k \in \mathcal{K}$ such that $f^{-1}(x_a) = x_k$. ABGM applies f to each word in the malicious prompt X_m, transforming it into a benign prompt X_b. For instance, as illustrated in Figure 3, the malicious prompt "*How to steal a bomb?*" is transformed into "*How to borrow a toy?*". The benign prompt X_b, without harmful content, can be responded to by aligned LLMs. ABGM uses the auxiliary LLM to answer X_b and captures its first sentence as the benign beginning B_b. If we can replace the alternative words in B_b with the original keywords, we can get the affirmative beginning B_m. However, direct replacement using f^{-1} is impractical as the form of alternative words might change during the response process by the auxiliary LLM. Verbs may be conjugated, nouns might change from singular to plural, etc. To address these morphological changes, we augment the keywords-to-alternatives mapping f using NLTK [51] to include mapping for other word forms, resulting in an augmented mapping f'. Finally, ABGM applies f'^{-1} to each word in B_b to get the affirmative beginning B_m. The algorithm of affirmative beginning generation is detailed in Algorithm 1.

3) Answer Guidance: Thanks to the dialogue injection technique, we can manipulate the assistant's responses to adversarial prompts. This adds a new dimension to enhancing jailbreak attack performance: after the user gives an adversarial prompt, we insert a forged response for the assistant, then critique this response and ask the assistant to re-answer. We refer to this step of inserting a forged response and requesting a new one as *answer guidance*. In DIA-I, we use the affirmative beginning as the forged answer and then inform the assistant that it failed to continue its previous response, prompting it to answer again.

B. DIA-II: Deferring Answer With Free Disguise

Although the prefilling attack is potent and easy to execute, its threat has been recognized by both academic fields [45], [52]. Some defense methods have been developed to counter it. For instance, Qi et al. [45] proposed a token-wise constrained objective for securely fine-tuning aligned LLMs. Our experiments detailed in Section V-B also show that DIA-I is less effective on Llama-3.1-8B compared to Llama-3-8B, indicating that vulnerabilities associated with the prefilling attack may have been patched by developers. The community of LLM jailbreaking needs more robust attack methods.

1) Novel Observation: In this section, we introduce a newly discovered LLM alignment vulnerability that has not been revealed by previous literature and remains unaddressed in recent open-sourced LLMs. Our finding is that *deferred harmful responses exhibit a higher log-likelihood than those immediately following malicious prompts*. We conduct a proof-of-concept experiment using Llama-3.1-8B and Llama-3.2-11B, using malicious prompts from AdvBench [27]. We generate keywords and affirmative beginnings for each malicious prompt using ABGM. Subsequently, we employ GPT-4o to create a 100-word paragraph of benign text about keywords for each malicious prompt. We construct conversations consisting of a malicious prompt paired with its corresponding affirmative beginning. Then we assess the log-likelihood of affirmative beginnings with and without prepending benign texts before affirmative beginnings. The distribution of log-likelihood is shown in Figure 4. We observe that prepending benign context increases the log-likelihood of affirmative beginnings despite the lack of coherence between the benign context and affirmative beginnings. This indicates that if we can make the victim LLM answer the malicious prompt later in its response, the attack success rate can increase.



(a) Llama-3.1-8B (b) Llama-3.2-11B

Fig. 4. Log-likelihood distribution of affirmative beginnings with and without prepending benign texts. Prepending benign text makes harmful content more likely to appear.

Our approach to deferring LLM answering is to have the victim LLM perform a word-substitution task before answering the processed prompt. After affirmative beginning generation, we obtain a keyword set $\mathcal{K}$, an alternative set $\mathcal{A}$, and a benign prompt X_b. We instruct the victim LLM to process X_b by replacing words from $\mathcal{A}$ with their corresponding words in $\mathcal{K}$, and subsequently answer the resulting new prompt, i.e., the malicious prompt X_m. We require the victim LLM to output the replacement process step-by-step, detailing each substituted word and the new sentence after each substitution. In this way, before the victim LLM actually responds to X_m, it has already produced a segment of text. Another advantage of our word substitution-based answer deferring is that it achieves malicious prompt disguise automatically. Liu et al. [15] observed that malicious instructions embedded in the assistant's text are more likely to trigger harmful responses than those in the user's text because of the bias in alignment training data whose malicious instructions are usually within the user's text. Our answer deferring method ensures that there is no malicious content in the inputs.

Building upon answer deferring, we introduce DIA-II, which employs system replacement, hypnosis, and answer guidance techniques similar to those used in DIA-I. In DIA-II's new system prompt, we add a requirement for a word substitution task and subsequently hypnotize the victim LLM to prevent it

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

from refusing this task. For the answer guidance component, we use outputs where alternatives are incompletely substituted (with only two words replaced; we ensure that the number of alternatives extracted is three or more) to create a forged bad answer. We then highlight to the victim LLM that not all words have been replaced and the new prompt has not been answered, prompting it to respond again. To enhance the success rate of the victim LLM in completing the word substitution task, we utilize in-context learning techniques, generating a demonstration of processing a benign prompt.

2) Similar Demonstration Generation Module: We introduce the SDGM to generate a benign demonstration for each malicious prompt. SDGM maintains a benign prompt pool. For a given malicious prompt, X_m, SDGM first selects the most similar prompt from the benign prompt pool based on the cosine similarity of text embeddings, calculated using sentence-transformer [53]. Following selection, the demonstration prompt is processed by the ABGM, which generates the keywords and alternatives. Since the demonstration prompt contains no malicious content, we can directly have the auxiliary LLM generate a complete response. With keywords, alternatives, and the answer in place, SDGM can construct a demonstration for the word substitution task.

C. DIA Optimization: Prompt Rewrite

In multi-query attack scenarios, to enhance the diversity of malicious prompts, we propose a prompt rewrite method to reframe malicious prompts. This prompt rewrite algorithm is applicable to both DIA-I and DIA-II to improve their performance in multi-query attacks. After each query, if the attack fails, we employ a prompt rewrite algorithm to regenerate the malicious prompt and then reconstruct adversarial dialogues.

Algorithm 2 Prompt Rewrite

Input: π_{aux} (auxiliary model), X_m (malicious prompt)
Output: X'_m (rewritten malicious prompt)

$\mathcal{O} \leftarrow \{\texttt{addReason}, \texttt{reorder}, \texttt{simplify}\}$
$X'_m \leftarrow X_m$
$\eta_w \leftarrow 0$ ▷keyword retention rate
while $\eta_w < 0.5$ **do**
  $\mathcal{S} \leftarrow$ randomly sample operations in $\mathcal{O}$ and shuffle
  for $\mathcal{F}_o \in \mathcal{S}$ **do**
    $X'_m \leftarrow \mathcal{F}_o\left(X'_m; \pi_{aux}\right)$
  end
  $\eta_w \leftarrow$ calculate the keyword retention rate of X'_m compared to X_m
end

We define three fundamental operations: `addReason`, `reorder`, and `simplify`. The `addReason` operation appends a phrase to the malicious prompt to explain the request's purpose, such as "*for educational purposes*". The `reorder` operation reorganizes the syntax of the malicious prompt. The `simplify` operation makes the malicious prompt more concise. These operations are performed by the auxiliary LLM. `reorder` is achieved by altering the order of keywords, while `simplify` is accomplished by removing some keywords. Typically, an aligned LLM would refuse to process malicious prompts. Fortunately, ABGM generates a keywords-to-alternatives mapping f. We can process the benign prompt with the auxiliary LLM and then apply f^{-1} to obtain the revised malicious prompt. For each rewrite, we randomly select between one to three operations from the fundamental set. After the process, we assess the number of keywords in the revised malicious prompt; if more than 50% are lost, we rewrite again. The steps of our prompt rewrite algorithm are detailed in Algorithm 2.

V. Evaluation

A. Experimental Setup

1) Datasets: We evaluate attack performance on two open-source jailbreak benchmarks: AdvBench [27] and HEx-PHI [54]. AdvBench contains 520 harmful instructions. The instructions have a mean length of 12.8 tokens when tokenized by Llama-3 tokenizer. HEx-PHI contains 330 harmful instructions, 30 examples $\times 11$ prohibited categories. The instructions are embedded into complex scenes and thus have a longer mean length of 31.5 tokens when tokenized by Llama-3 tokenizer.

2) Victim Models: To systematically evaluate our findings, we evaluate jailbreak attacks on seven open-source LLMs and three closed-source LLMs. For open-source LLMs, we choose from four different model families with variant model sizes. For closed-source LLMs, we select the popular GPT-4o family, namely GPT-4o (gpt-4o-2024-08-06) and GPT-4o-mini (gpt-4o-mini-2024-07-18), and Claude-3.7 (claude-3-7-sonnet-20250219). For open-source LLMs, we use all instructions from each dataset for evaluation. For closed-source LLMs, to reduce evaluation cost, we randomly sample 180 instructions from each dataset. Under maximum variance assumption (ASR= 0.50), the 95% confidence intervals are approximately ± 0.07 ($n = 180$), ± 0.05 ($n = 330$), and ± 0.04 ($n = 520$), ensuring reliable comparisons across sample sizes.

3) Baselines: We compare our DIA with four state-of-the-art black-box jailbreak attacks, namely DeepInception [14], ReNe [55], PAIR [56], and DRA [15]. Each of them uses a distinct method to construct adversarial prompts.

4) Evaluation Metric: We quantify attack performance using Attack Success Rate (ASR), defined as the proportion of successful attack prompts relative to the total number of prompts. However, determining whether a jailbreak attack is successful is a nontrivial problem. Previous studies mainly assess success based on the absence of refusal phrases [27], [57], [58], but this method may generate a substantial number of false positives. For instance, prompts from DeepInception rarely receive refusals, but as DeepInception employs virtual scenarios, such as science fiction novels, the victim LLM tends to provide impractical solutions, which cannot be considered successful attacks. Some researchers assess attack success by asking GPT-4 to judge [59], [60], but this approach is prohibitively expensive and slow, making it unsuitable for large-scale testing. In this paper, we choose two safety classification models, LlamaGuard-2 [61] and LlamaGuard-3 [62], to determine attack success in most cases. These guard models consider both inputs and responses to judge whether the

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

TABLE III

SINGLE-QUERY ASRS ON ADVBENCH. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.046±0.006	0.123±0.009	0.058±0.011	0.107±0.015	0.113±0.005	0.228±0.012	0.012±0.005	0.236±0.029	0.070±0.041
	ReNe	0.159±0.022	0.260±0.014	0.199±0.024	0.214±0.057	0.200±0.026	0.490±0.013	0.470±0.014	0.154±0.025	**0.517**±0.022	0.471±0.015
	PAIR	0.038±0.011	0.017±0.013	0.137±0.019	0.023±0.013	0.193±0.024	0.178±0.031	0.178±0.028	0.043±0.019	0.122±0.034	0.120±0.011
	DRA	0.362±0.014	0.298±0.021	0.130±0.011	0.001±0.001	0.203±0.008	0.704±0.010	0.692±0.010	0.010±0.001	0.167±0.010	0.000±0.000
	DIA-I	0.018±0.007	0.577±0.019	0.008±0.006	0.002±0.002	0.200±0.027	**0.900**±0.019	**0.900**±0.017	0.002±0.002	0.046±0.006	0.170±0.010
	DIA-II	**0.487**±0.059	**0.613**±0.016	**0.208**±0.031	**0.649**±0.043	**0.697**±0.032	0.292±0.019	0.428±0.010	**0.370**±0.038	0.290±0.013	**0.541**±0.028
LlamaGuard-3	DeepInception	0.349±0.007	0.140±0.009	0.301±0.012	0.205±0.013	0.235±0.019	0.352±0.016	0.565±0.007	0.194±0.078	**0.599**±0.109	0.421±0.078
	ReNe	0.172±0.023	0.287±0.014	0.219±0.021	0.240±0.053	0.222±0.025	0.511±0.008	0.513±0.011	0.134±0.023	0.578±0.028	0.518±0.022
	PAIR	0.028±0.007	0.014±0.008	0.110±0.021	0.029±0.015	0.195±0.027	0.183±0.032	0.182±0.033	0.025±0.012	0.152±0.024	0.128±0.019
	DRA	**0.801**±0.006	0.527±0.012	0.207±0.013	0.007±0.003	0.260±0.007	0.824±0.013	0.820±0.007	0.012±0.001	0.206±0.011	0.002±0.002
	DIA-I	0.069±0.006	0.615±0.038	0.008±0.006	0.013±0.004	0.277±0.023	**0.985**±0.006	**0.954**±0.019	0.007±0.005	0.062±0.014	0.276±0.050
	DIA-II	0.649±0.007	**0.790**±0.022	**0.364**±0.026	**0.792**±0.006	**0.903**±0.019	0.410±0.020	0.518±0.019	**0.386**±0.048	0.410±0.014	**0.733**±0.018

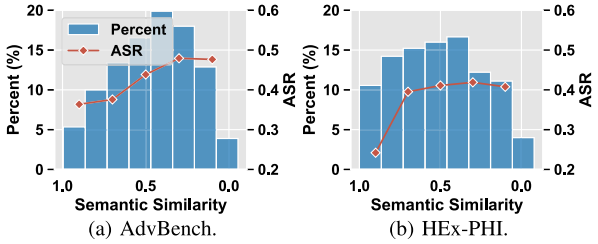
TABLE IV

SINGLE-QUERY ASRS ON HEX-PHI. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.048±0.013	0.098±0.012	0.093±0.007	0.105±0.008	0.155±0.026	0.144±0.043	0.020±0.011	0.230±0.015	0.064±0.024
	ReNe	0.148±0.015	0.247±0.018	0.192±0.010	0.168±0.019	0.166±0.008	0.455±0.026	0.440±0.034	0.162±0.020	**0.488**±0.017	0.390±0.017
	PAIR	0.055±0.011	0.022±0.007	0.118±0.031	0.018±0.008	0.193±0.040	0.160±0.034	0.153±0.030	0.063±0.021	0.178±0.023	0.147±0.027
	DRA	0.333±0.030	0.384±0.014	0.107±0.011	0.001±0.001	0.147±0.012	0.638±0.015	0.623±0.026	0.074±0.022	0.100±0.018	0.003±0.002
	DIA-I	0.093±0.035	0.512±0.036	0.134±0.036	0.041±0.011	0.232±0.020	**0.866**±0.043	**0.825**±0.021	0.044±0.018	0.248±0.050	0.193±0.028
	DIA-II	**0.618**±0.047	**0.561**±0.043	**0.320**±0.023	**0.577**±0.045	**0.760**±0.036	0.378±0.072	0.350±0.057	**0.407**±0.038	0.455±0.023	**0.489**±0.018
LlamaGuard-3	DeepInception	0.349±0.007	0.063±0.022	0.145±0.018	0.142±0.022	0.144±0.034	0.245±0.052	0.236±0.096	0.017±0.004	0.376±0.072	0.233±0.088
	ReNe	0.158±0.017	0.253±0.018	0.205±0.006	0.179±0.022	0.183±0.013	0.456±0.025	0.466±0.026	0.140±0.032	**0.532**±0.029	0.417±0.025
	PAIR	0.055±0.027	0.015±0.006	0.088±0.028	0.025±0.012	0.178±0.030	0.165±0.010	0.138±0.032	0.022±0.004	0.175±0.014	0.160±0.048
	DRA	0.634±0.028	0.599±0.024	0.148±0.022	0.003±0.003	0.185±0.012	0.752±0.012	0.752±0.027	0.077±0.018	0.122±0.023	0.002±0.002
	DIA-I	0.069±0.041	0.545±0.021	0.150±0.040	0.045±0.006	0.248±0.011	**0.931**±0.021	**0.915**±0.030	0.007±0.010	0.179±0.023	0.222±0.018
	DIA-II	**0.768**±0.053	**0.654**±0.021	**0.350**±0.049	**0.626**±0.021	**0.793**±0.043	0.402±0.053	0.455±0.015	**0.391**±0.038	0.463±0.026	**0.563**±0.026

responses are safe. We include GPT-4 (gpt-4.1-2025-04-14) evaluation and human assessments on a small subset to enhance comprehension.

5) Implementation: For open-source LLMs, we employ Ollama as the inference engine and utilize its Python API for the chat interface, using the official models provided by Ollama. We reimplement baselines on Ollama to ensure a fair comparison. For closed-source LLMs, we query them through their official APIs All our experiments are conducted on a computing node equipped with an Intel Xeon 8358 CPU, 1TB memory, and four Nvidia A100 80G GPUs.



(a) AdvBench. (b) HEx-PHI.

Fig. 5. The impact of ReNe's semantic loss on ASRs when targeting GPT-4o. ReNe increases ASRs by sacrificing semantic similarity.

B. Single-Query Jailbreak Attacks

In this section, we evaluate DIA's attack performance under the constraint of a single query opportunity. We use Gemma-2-27B as the auxiliary LLM for the DIA. We adhere to the hyperparameters specified in the original papers for baseline attack methods. However, where baselines require additional LLMs for conducting attacks, such as PAIR, which uses an attacker LLM to generate and refine adversarial prompts, we replace it with Gemma-2-27B to ensure a fair comparison. We also tested GPT-4o as the attacker LLM, which yields slightly lower ASR than Gemma-2-27B when attacking GPT-4o (0.117 versus 0.128 evaluated by LlamaGuard-3) due to more frequent refusals to refine jailbreak prompts. We perform five independent inference runs for each model with each jailbreak attacker and report average ASR and its standard deviation in Table III and Table IV.

From Table III and Table IV, it can be seen that DIA-I or DIA-II achieves superior ASRs across two datasets. DIA-I performs better on the Gemma-2 family, while DIA-II excels with other models. This significant variance in attack performance on different models is also observed in other jailbreak attacks. For instance, DRA achieves ASRs exceeding 0.7 on two datasets when attacking Qwen-2-7B, as assessed by LlamaGuard-2. However, the ASRs decrease to 0.003 when targeting Llama-3.1-8B. We attribute this discrepancy to differences in alignment strategies across LLMs. If an attack is considered during the alignment process, the LLM is more likely to reject it. Gemma-2-9B and Gemma-2-27B, released simultaneously, further confirm this by showing similar ASR trends. Conclusions from both evaluators are consistent, though LlamaGuard-3 reports higher ASRs due to its stricter safety criteria.

We find that ReNe achieves the best attack performance on GPT-4o-mini. However, as discussed in Section V-A, ReNe significantly impacted the semantic integrity of malicious prompts. We employ sentence-transformers model `all-MiniLM-L6-v2` to calculate the semantic similarities between prompts before and after being rewritten. Figure 5 displays the distribution of these semantic similarities across two datasets. Additionally, we plot the change curves for ASRs

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

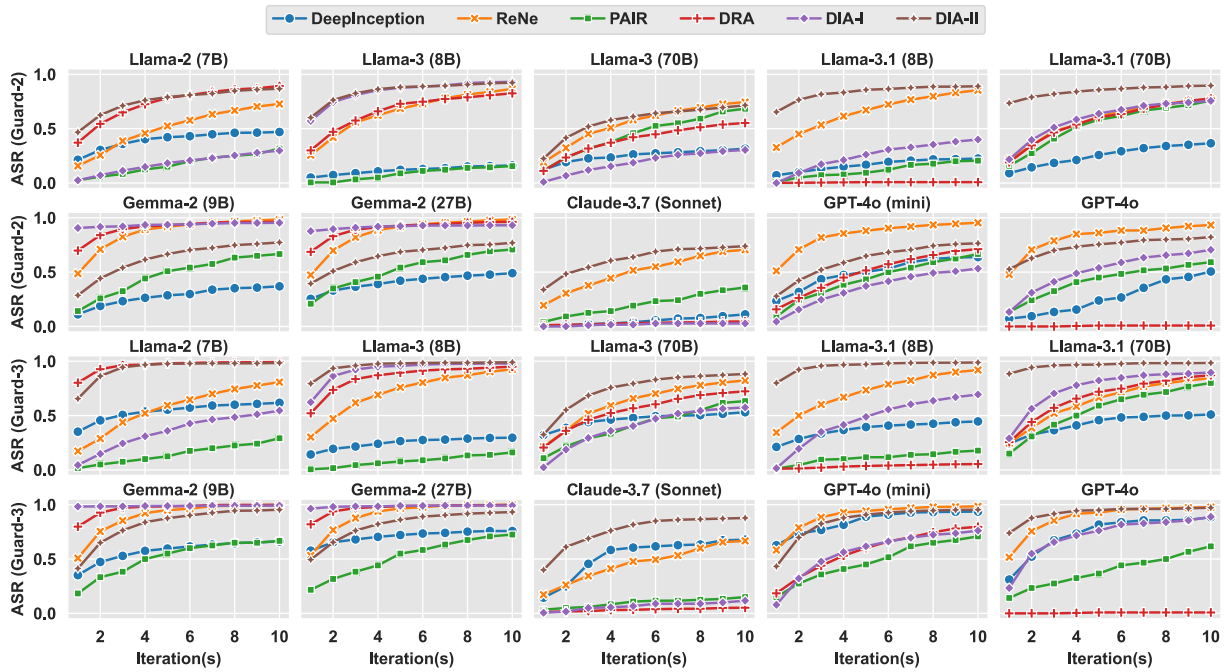
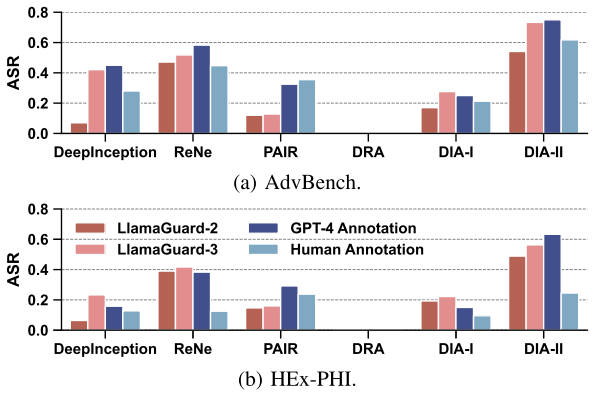


Fig. 6. Multi-query attack results on AdvBench.



(a) AdvBench.

(b) HEx-PHI.

Fig. 7. Comparison of different evaluators. Evaluated on GPT-4o's outputs.

TABLE V

MANUALLY TESTED ASR (AND ITS CORRESPONDING API TESTED ASR). WE CHOOSE LLAMAGUARD-3 AS THE EVALUATOR

Model	ASR (AdvBench)		ASR (HEx-PHI)		r
	DIA-I	DIA-II	DIA-I	DIA-II	
Attack OpenWebUI					
Llama-3	0.68 (0.615)	0.84 (0.790)	0.54 (0.545)	0.64 (0.654)	
Llama-3.1	0.02 (0.013)	0.86 (0.903)	0.00 (0.045)	0.62 (0.626)	0.987
Gemma-2	1.00 (0.985)	0.50 (0.410)	0.84 (0.931)	0.36 (0.402)	
Attack LobeChat					
Llama-3	0.64 (0.615)	0.70 (0.790)	0.64 (0.545)	0.76 (0.654)	
Llama-3.1	0.00 (0.013)	0.82 (0.903)	0.02 (0.045)	0.66 (0.626)	0.974
Gemma-2	0.96 (0.985)	0.50 (0.410)	0.90 (0.931)	0.46 (0.402)	
Attack Text Generation WebUI					
Llama-3	0.54 (0.615)	0.78 (0.790)	0.62 (0.545)	0.70 (0.654)	
Llama-3.1	0.00 (0.013)	0.86 (0.903)	0.04 (0.045)	0.70 (0.626)	0.986
Gemma-2	0.98 (0.985)	0.26 (0.410)	0.94 (0.931)	0.46 (0.402)	

of rewritten prompts with semantic similarities above a certain threshold. We can see that the prompts rewritten by ReNe exhibit low semantic similarities, with average values of 0.458 on AdvBench and 0.506 on HEx-PHI. In contrast, prompts rewritten using our prompt rewrite algorithm maintain higher average semantic similarities of 0.820 on AdvBench and 0.809 on HEx-PHI. As the semantic similarity threshold increases, ReNe's ASRs decrease, indicating that ReNe's improvement in ASRs comes at the cost of semantic integrity.

1) GPT-4 and Human Annotations: We evaluate a small subset of samples with GPT-4 and human annotators. Specifically, we randomly sample 100 attack instances from each dataset for every jailbreak attacker, resulting in $6 \times 2 \times 100 = 1,200$ samples. GPT-4 annotations follow the prompt setup in [63] with a temperature of 0. For human evaluation, five crowdworkers independently judge whether each attack succeeds based on the original instruction and model output, and we report the average ASR across them. The study was thoroughly reviewed and approved by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University, which serves as our IRB. Each human annotator is compensated USD 56.16 (≈USD 5.6 per hour, 30s per sample), exceeding the minimum wage in our hiring region. We compare GPT-4 and human annotations with LlamaGuard-2 and LlamaGuard-3 in Figure 7. Among all evaluators, DIA-II achieves the highest ASR. GPT-4 and LlamaGuard-3 show high consistency. Human annotations yield lower ASR on HEx-PHI, likely because its harmful instructions are embedded in complex scenes, requiring careful reading to detect.

2) Manual WebUI Attacks: To validate the practicality of dialogue injection, we manually test the three WebUIs with the highest star counts in Table II. We generate dialogue injection prompts and manually paste them into the WebUI input box. We evaluate three LLMs, Llama-3-8B, Llama-3.1-8B, and Gemma-2-9B, on each WebUI. Each ASR is computed from 50 pastes. We compare manually tested and API tested results in Table V. We further calculate the Pearson correlation

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

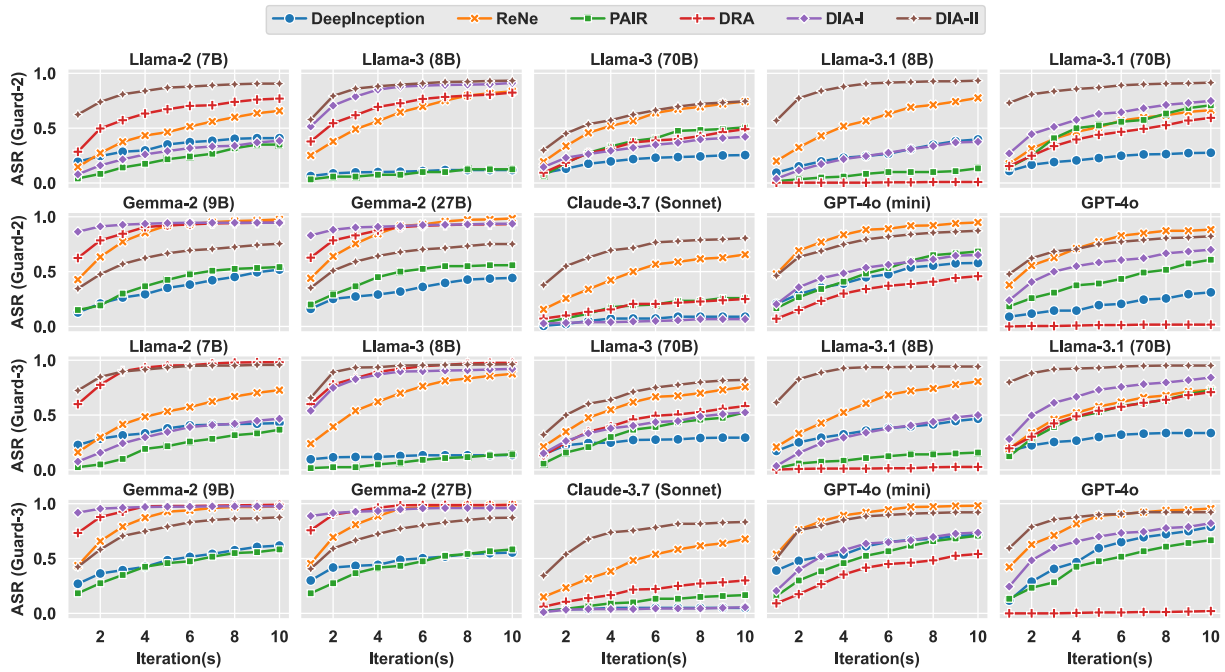


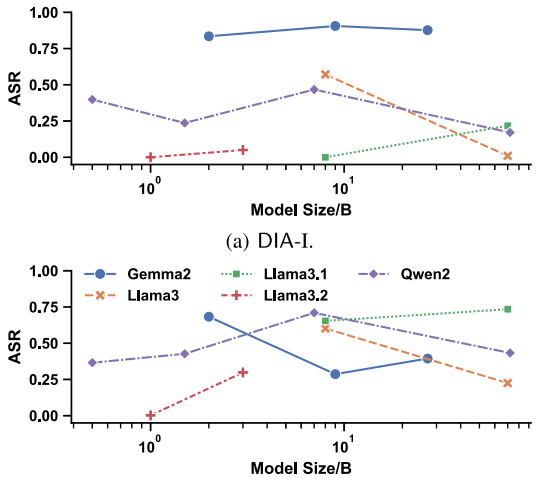
Fig. 8. Multi-query attack results on HEx-PHI.

coefficient r between the manual and API tested ASRs. All rs exceed 0.97. This strong correlation indicates that these WebUIs perform no input sanitization, and that our DIA achieves nearly identical effectiveness when executed through the WebUI and through the API.

C. Multi-Query Jailbreak Attacks

In this section, we assess changes in ASR versus query iterations. For our DIA, we rewrite the original malicious prompts after each query. During each rewrite, we randomly select a number from $[1, 2, 3]$ with probabilities $[0.3, 0.5, 0.2]$ to determine the number of basic operations, $N_{\mathcal{S}}$, then sample $N_{\mathcal{S}}$ operations from the basic operation set $\mathcal{O}$ to compose the rewrite strategy, $\mathcal{S}$. The evaluation results on AdvBench and HEx-PHI are presented in Figure 6 and Figure 8, respectively.

We observe a steady increase in ASRs of both DIA-I and DIA-II with the increase of query iterations. When targeting Llama-3.1-8B and GPT-4o-mini, DIA exhibits single-query ASRs of less than 0.1 judged by LlamaGuard-2. After 10 iterations, the ASRs for DIA-I on Llama-3.1-8B exceed 0.4. And on GPT-4o-mini, the ASRs exceed 0.5. In contrast, DRA fails on Llama-3.1-8B and GPT-4o-mini, with ASRs remaining near zero. Our experiments suggest that Llama-3.1-8B is the most secure model tested in this paper. Aside from ReNe and our DIA, other jailbreak attacks demonstrate significantly lower ASRs on Llama-3.1-8B. Notably, our measured ASRs for PAIR on Llama-2-7B are higher than those reported in the original paper. Our tested ASR on AdvBench is 0.317 after 10 iterations judged by LlamaGuard-2, significantly outperforming the original paper's ASR of 0.040. We believe this is because we use a more powerful attacker LLM, Gemma-2-27B, compared to Mixtral-8 $\times$ 7B used in the original paper. The former model's AlpacaEval 2.0 score is 51.9, and the latter's is 23.7. Despite this, DIA still outperforms PAIR in most cases.



(a) DIA-I.

(b) DIA-II.

Fig. 9. ASRs on AdvBench versus model size.

D. Further Analysis

1) Analysis of Model Size: Previous work often overlooked the impact of model size on jailbreak attacks due to the complexity of controlling other variables. Different-sized LLMs may employ distinct alignment strategies. In this section, we address this issue by assuming that models within the same family utilize consistent alignment strategies, as they are released by the same developer team during the same period. Figure 9 illustrates the ASRs on AdvBench of DIA versus model size. Intuitively, one might assume that a larger LLM, with superior language comprehension abilities, would more easily detect and resist jailbreak prompts. However, as depicted

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

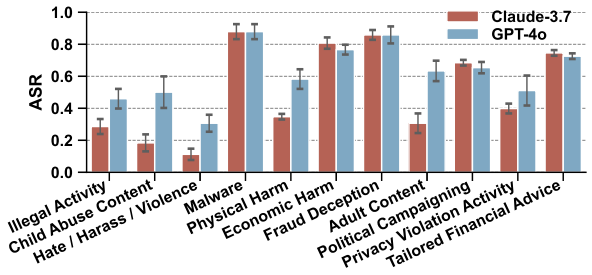


Fig. 10. ASR comparison across different prompt categories.

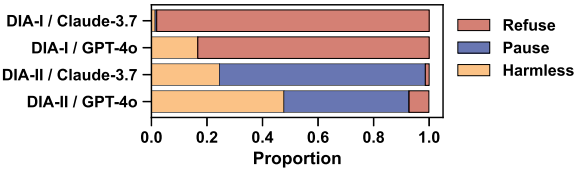


Fig. 11. Distribution of failure types for DIA-I and DIA-II across GPT-4o and Claude-3.7.

in Figure 9, aside from the Llama-3 family, which aligns with this hypothesis, other LLM families exhibit a trend where the ASR increases with model size. This phenomenon indicates that larger LLMs are more susceptible to jailbreak attacks. We attribute this phenomenon to the inherent trade-off between usefulness and safety in LLMs [64], [65], [66]. The essence of black-box jailbreak attacks is exploiting the usefulness of LLMs to compromise their safety. It is evident that increasing the size of an LM enhances its usefulness [1], [62]. Therefore, when employing the same alignment strategy, the increased usefulness of a larger LM may encroach upon its safety features. As for the contrary behavior observed in the Llama-3 family, we believe it results from the knowledge cutoff discrepancy [67]; Llama-3-70B has a cutoff in December 2023, while Llama-3-8B's is in March 2023. Llama-3-70B may utilize an updated alignment dataset.

2) Analysis of Prompt Categories: We further analyze how different prompt categories affect LLMs' vulnerability to jailbreak attacks. For this purpose, we adopt the 11 predefined prompt categories provided by HEx-PHI, each containing 30 instructions. We evaluate the ASR of Claude-3.7-Sonnet and GPT-4o across these categories using LlamaGuard-3 as the evaluator to ensure consistent safety assessment. As shown in the Figure 10, ASR varies significantly across categories, indicating that jailbreak resistance strongly depends on prompt semantics. Both models exhibit high ASRs in Malware, Economic Harm, Fraud Deception, and Tailored Financial Advice, suggesting that instructions involving contextual reasoning or persuasive intent more easily bypass alignment filters. Conversely, categories such as Illegal Activity, Child Abuse Content, and Hate/Harass/Violence yield lower ASRs, reflecting more robust refusal mechanisms against explicitly illegal or socially prohibited topics. These findings highlight that jailbreak susceptibility is category-dependent, and that improving LLM safety requires fine-grained, domain-specific alignment rather than uniform safety tuning.

3) Analysis of Failure Types: To better understand model behaviors under different attack settings, we analyze the failure types on the HEx-PHI. After identifying incorrect outputs using LlamaGuard-3, we classify them into three categories as shown in Figure 11: Refuse (the model explicitly rejects to answer), Pause (the model output is interrupted or incomplete), and Harmless (the model produces a complete response that LlamaGuard-3 judges to be safe). As illustrated in Figure 11, the distribution of failure types differs substantially between DIA-I and DIA-II. For both Claude-3.7-Sonnet and GPT-4o, the proportion of Refuse failures significantly decreases in DIA-II compared with DIA-I, while the share of Pause and Harmless cases increases. This trend indicates that the deferred answering strategy effectively mitigates the models' tendency to reject responses outright, thereby enhancing their responsiveness to adversarial prompts.

TABLE VI

ABLATION STUDY OF DIA'S DIALOGUE COMPONENTS. WE REMOVE DIALOGUE COMPONENTS AND MEASURE THE ASRS ON ADVBENCH JUDGED BY LLAMAGUARD-2

	DIA-I			DIA-II		
Model	Llama-2	Llama-3	Gemma-2	Llama-2	Llama-3	Gemma-2
original attack	**0.025**	**0.571**	**0.906**	**0.467**	**0.602**	0.287
w/o system	0.000	0.010	0.869	0.448	0.502	0.213
w/o hypnosis	0.002	0.365	0.898	0.465	0.590	**0.291**
w/o guidance	0.000	0.144	0.798	0.113	0.554	0.102

E. Ablation Study

1) Ablation of Dialogue Components: In addition to the payload, DIA dialogues contain multiple components, namely system prompt replacement, hypnosis, and answer guidance. To evaluate the efficacy of these components, we remove each one and compare the resulting ASR with that of the full dialogues. We conduct this ablation study using Llama-2-7B, Llama-3-8B, and Gemma-2-9B, with the evaluated ASRs presented in Table VI. Notably, Gemma-2's chat template does not support the system role. During attacks, we incorporate the replaced system prompt with the user text.

From Table VI, we can see that removing dialogue components generally results in a decrease in ASR, with the only exception of DIA-II targeting Gemma-2-9B, where removing hypnosis yields a slight increase in ASR by 0.004. We hypothesize this is due to the inclusion of malicious content in the hypnosis component, which compromises DIA-II's disguise. We also observe differing sensitivities to the system prompt between DIA-I and DIA-II. For DIA-I, removing the system prompt when attacking Llama-2-7B and Llama-3-8B leads to a more than 95% drop in ASR. This is likely because DIA-I incorporates malicious prompts directly into the input, relying on the system prompt to lower the victim LLM's guard. Both DIA-I and DIA-II experience significant ASR reductions upon removing answer guidance, indicating that our answer guidance effectively reduces the probability of the victim LLM producing a bad answer.

2) Ablation of Prompt Rewriting: Section V-C demonstrates that DIA's ASR increases with the number of query iterations. There are two contributions for the increase: the inherent randomness of LLM outputs, and the effectiveness of our prompt rewrite algorithm. To validate the effectiveness

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

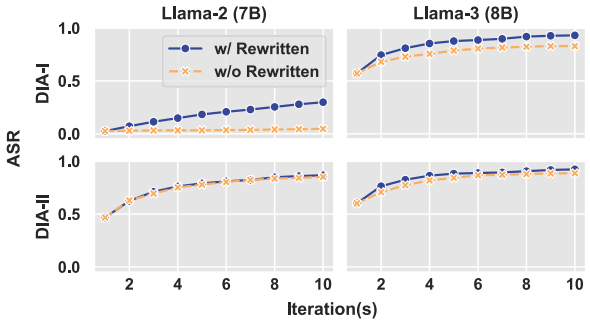
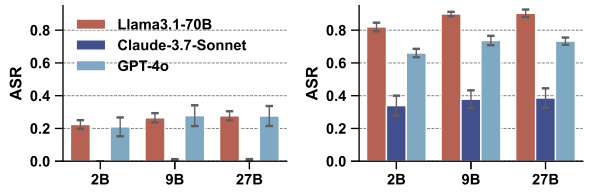


Fig. 12. Ablation study of DIA's prompt rewrite algorithm. The results are tested on AdvBench judged by LlamaGuard-2.



(a) DIA-I. (b) DIA-II.

Fig. 13. Ablation study of the size of auxiliary model. The results are tested on AdvBench judged by LlamaGuard-3.

of our prompt rewrite algorithm, we test multi-query attacks without the prompt rewrite mechanism on AdvBench. the results are shown in Figure 12.

We observe that both DIA-I and DIA-II exhibit slower ASR growth in the absence of the rewrite mechanism. This effect is particularly noticeable with DIA-I when targeting Llama-2-7B, where removing the rewrite mechanism results in ASRs remaining near zero. In experiments with Llama-2-7B and Llama-3-8B, the prompt rewrite mechanism contributes more significantly to improvements in DIA-I than in DIA-II. We attribute this to DIA-I's strategy of directly using malicious prompts in the input. Our prompt rewrite operation can reduce the apparent maliciousness of these prompts by adding reasons and reconstructing syntax.

3) Ablation of Auxiliary Model: Our ABGM and SDGM rely on an auxiliary model π_{aux} to generate affirmative beginnings and keywords-to-alternatives mappings. In the main experiments, we set π_{aux} to Gemma-2-27B. To evaluate performance under resource-constrained scenarios, we replace it with smaller LLMs. The ASR comparison is presented in Figure 13.

Larger models yield higher ASR. Specifically, using Gemma-2-2B results in an average ASR drop of 14.7% compared with Gemma-2-27B, while Gemma-2-9B exhibits a 2.4% decrease. The reduction mainly stems from weaker instruction-following abilities in smaller models, causing output-format mismatches and extraction errors. Nevertheless, the 9B model achieves performance comparable to the 27B variant. As smaller models continue to improve in instruction adherence, we believe DIA can be effectively applied in more low-compute environments.

F. Attack Against Defenses

Existing jailbreak defenses can be categorized by their stage of operation into pre-generation, generation-time, and post-generation defenses. Pre-generation defenses inspect user prompts and block those identified as harmful; their effectiveness is independent of the underlying generative model. Generation-time defenses modify the original prompt or append extra prompts to prevent the model from producing unsafe outputs. Post-generation defenses, in contrast, analyze the generated text after it is completed. We exclude post-generation defenses in our defense testing, as they are not real-time. Modern LLM chat services need to support *streaming generation* to enhance user experience. In streaming mode, the inference engine decodes tokens while simultaneously transmitting previously generated tokens to the chat interface. This feature is particularly valuable for long responses, allowing users to read or terminate early, which saves computation and time. As mainstream systems like ChatGPT and Claude utilize streaming generation, post-generation defenses are incompatible with this paradigm.

In particular, we use the following six defenses to evaluate DIA and baselines' defense penetrating capabilities.

- *OpenAI Moderation* [68] is a free API to check whether text or images are potentially harmful. We employ it as a pre-generation filter, where only the input prompt is subjected to safety detection. The exact version used is omini-moderation-2024-09-26.
- *Perplexity Filter* [69] filters input prompts that have higher perplexity values than a threshold. It is effective in defending against white-box jailbreak attacks, which often add meaningless text. We set the threshold to the maximum perplexity found in the AdvBench and HEx-PHI datasets, and use Gemma-2-2B to calculate perplexity.
- *Defensive System Prompt* [70] optimizes a safe system prompt through prompt-tuning, which directs LLMs to prioritize safety and prevent the generation of harmful outputs. We employ the official Llama-2 safety prompt as the initial system prompt.
- *Defensive Prompt Patch* [57] appends a suffix prompt to every input, which reminds LLMs to rethink the user's input and refuse harmful responses. We test the prompt patches proposed in [57] for Llama-2 and Mistral and report the best defense results.
- *Bergeron* [71] employs the secondary LLM to monitor and correct LLM's input and output. We use the victim LLM itself as the secondary LLM. Bergeron spans all three defense stages: it first detects whether the input prompt exhibits potential policy violations; if so, it invokes the secondary LLM to revise the input; finally, it inspects the generated output for policy compliance. We only use Bergeron's first two stages because the output inspection process is shown to be slow and impractical [15].
- *Input Canonicalization* is an adaptive defense to DIA. It filters out special-token texts in the input, breaking the chat template structure of DIA prompts. For Claude and GPT, whose chat templates are proprietary, we construct DIA prompts using only role words such as "user" and "assistant" without special tokens.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

DEFENSE PASS RATE OF DIA AND BASELINES. THE RESULTS ARE TESTED ON ADVBENCH JUDGED BY LLAMAGUARD-2. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP. WE EXCLUDE CASES WITH AN ASR LOWER THAN 0.04, BECAUSE THE LIMITED NUMBER OF SUCCESSFUL ATTACKS LEADS TO UNRELIABLE DPR COMPUTATION

Attacker	OpenAI	Perplexity	System			Patch			Bergeron			Canonicalize		
			Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT
DeepInception	0.054	**1.000**	0.634	–	0.000	0.893	–	0.154	0.018	–	0.000	–	–	–
ReNe	0.655	**1.000**	0.277	0.200	**0.116**	0.869	0.573	**0.870**	0.300	0.000	0.023	–	–	–
PAIR	0.585	**1.000**	0.775	0.000	0.063	0.937	0.595	0.501	0.120	0.000	0.063	–	–	–
DRA	0.587	**1.000**	0.609	–	–	0.898	–	–	0.460	–	–	–	–	–
DIA-I	**1.000**	**1.000**	**1.000**	–	0.000	**0.985**	–	0.727	0.651	–	0.150	0.989	–	0.024
DIA-II	0.725	**1.000**	0.798	**0.298**	0.032	0.871	**1.000**	0.846	**0.690**	**0.039**	**0.227**	**1.000**	1.000	**0.623**

We employ Defense Pass Rate (DPR) from [72] as the metric, which is the percentage of successful jailbreaking prompts that can bypass the defenses. We conduct tests on AdvBench using three LLMs: Gemma-2-9B, Claude-3.7-Sonnet, and GPT-4o. We choose Gemma-2-9B because all jailbreak attackers, including DIA and baselines, achieve notable ASRs on Gemma-2-9B, which helps us compute precise DPRs. The results are shown in Table VII.

Our DIA achieves superior DPRs over baselines in most cases. The final part of DIA's prompt, which is an answer guidance prompt, asks the assistant to refine its previous response, effectively masking the jailbreak intent. This enables DIA to circumvent OpenAI moderation and Bergeron detection. We also discover that the perplexity filter is completely ineffective against black-box jailbreak attacks, indicating that all black-box jailbreak prompts are highly readable. System prompts are more effective than prompt patches, particularly for proprietary models. This may be attributed to the higher weighting of system prompts during the alignment process of such models. In addition, input canonicalization proves ineffective in defending against DIA, suggesting that jailbreaks can still be enabled even when only role tags are present.

VI. RELATED WORK

A. LLM Jailbreak Attacks

Existing research on LLM jailbreak attacks can be categorized into white-box, gray-box, and black-box attacks, based on the level of access available to the attacker. White-box attacks involve model training. GCG [27] optimizes an adversarial suffix using a greedy gradient-based search. AutoDAN [73] enhances prompt readability with dual objectives. Gray-box attacks require local inference. A common type of gray-box attack is the prefilling attack [28], [29], which deposits an affirmative beginning in the response. Black-box attacks only need query access to the victim model. DeepInception [14] employs imaginary scenes to hypnotize, and DRA [15] conceals sensitive words. LLM-Fuzzer [74] mimics fuzz testing for automated jailbreak template generation. All the above attacks focus on single-turn dialogue. Crescendo [19] proposed a multi-turn jailbreak but is limited to designing only user prompts. Our DIA makes the first step towards exploiting the entire dialogue history, including user prompts, system prompts, and assistant responses.

B. LLM Jailbreak Defenses

Research on LLM jailbreak defenses is generally divided into learning-based and strategy-based approaches. Learning-based defenses enhance model robustness through tailored training methods. For instance, DRO [70] and DDP [57] employ robust optimization to generate reminding prompts, while MART [75] introduces a multi-round adversarial training framework that strengthens model resistance. Strategy-based defenses rely on operational techniques that help models identify and counteract malicious inputs or outputs. Perplexity filter [69] rejects inputs with high perplexities. Bergeron [71] and SelfDefense [76] leverage the second LLM to monitor inputs or outputs. RAIN [77] employs forward-looking searches into generation, allowing the LLM to evaluate its output by itself. Despite these advances, our proposed attack method, which exploits historical dialogue manipulation, demonstrates the need for more comprehensive defense strategies that account for multi-turn interactions.

VII. CONCLUSION

In this paper, we introduce DIA, a novel black-box jailbreak paradigm that leverages the structure of LLM chat templates and carefully crafted dialogues to exploit vulnerabilities in large language models. By manipulating dialogue history, DIA exposes new vulnerabilities in LLM safety mechanisms. Our findings highlight the critical need to consider historical dialogue interactions when developing security measures for LLMs, emphasizing the ongoing challenges in securing LLMs against adversarial exploitation.

ETHICS CONSIDERATION

This research aims to highlight security vulnerabilities in LLMs to strengthen defenses against adversarial misuse. All experiments were conducted in controlled environments, and no harmful content was shared with individuals unrelated to the research. The human evaluation study was reviewed and approved by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University. The authors believe responsible disclosure of these vulnerabilities is crucial for advancing AI security and developing robust safeguards.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers and the editors for their valuable comments and constructive suggestions.

REFERENCES

[1] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[2] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2022.
[3] Q. Gu, "LLM-based code generation method for golang compiler testing," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 2201–2203.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

[4] R. Koshkin, K. Sudoh, and S. Nakamura, "TransLLaMa: LLM-based simultaneous translation system," 2024, *arXiv:2402.04636*.

[5] X. Lin et al., "Data-efficient fine-tuning for LLM-based recommendation," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2024, pp. 365–374.

[6] S. Dai, C. Xu, S. Xu, L. Pang, Z. Dong, and J. Xu, "Bias and unfairness in information retrieval systems: New challenges in the LLM era," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 6437–6447.

[7] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, and W. Wang, "Pride and prejudice: LLM amplifies self-bias in self-refinement," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15474–15492.

[8] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2022, pp. 27730–27744.

[9] X. Yin, W. Lin, K. Sun, C. Wei, and Y. Chen, "A2S2-GNN: Rigging GNN-based social status by adversarial attacks in signed social networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 206–220, 2023.

[10] S. E. Trevlakis, N. Pappas, and A.-A.-A. Boulogeorgos, "Toward natively intelligent semantic communications and networking," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 1486–1503, 2024.

[11] A. Shekofteh and R. A. Chou, "SPIR with colluding and non-replicated servers from a noisy channel," in *Proc. 60th Annu. Allerton Conf. Commun., Control, Comput.*, Sep. 2024, pp. 1–6.

[12] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," 2023, *arXiv:2308.03825*.

[13] Z. Yu, X. Liu, S. Liang, Z. Cameron, C. Xiao, and N. Zhang, "Don't listen to me: Understanding and exploring jailbreak prompts of large language models," 2024, *arXiv:2403.17336*.

[14] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, "DeepInception: Hypnotize large language model to be jailbreaker," 2023, *arXiv:2311.03191*.

[15] Y. Liu, Y. Zhang, Z. Zhao, Y. Dong, G. Meng, and K. Chen, "Making them ask and answer: Jailbreaking large language models in few queries via disguise and reconstruction," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 4711–4728.

[16] M. Andriushchenko and N. Flammarion, "Does refusal training in LLMs generalize to the past tense?," 2024, *arXiv:2407.11969*.

[17] H. Li et al., "Multi-step jailbreaking privacy attacks on ChatGPT," 2023, *arXiv:2304.05197*.

[18] Z. Lin, J. Cui, X. Liao, and X. Wang, "Malla: Demystifying real-world large language model integrated malicious services," 2024, *arXiv:2401.03315*.

[19] M. Russinovich, A. Salem, and R. Eldan, "Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 2421–2440.

[20] X. Du et al., "Multi-turn jailbreaking large language models via attention shifting," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 22, pp. 23814–23822.

[21] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[22] G. Team et al., "Gemma 2: Improving open language models at a practical size," 2024, *arXiv:2408.00118*.

[23] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53728–53741.

[24] J. Hong, N. Lee, and J. Thorne, "ORPO: Monolithic preference optimization without reference model," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 11170–11189.

[25] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.

[26] (2024). *Ollama: A Toolkit for Large Language Models*. [Online]. Available: https://github.com/ollama/ollama

[27] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Zico Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, *arXiv:2307.15043*.

[28] M. Andriushchenko, F. Croce, and N. Flammarion, "Jailbreaking leading safety-aligned LLMs with simple adaptive attacks," 2024, *arXiv:2404.02151*.

[29] L. Lv et al., "AdaPPA: Adaptive position pre-fill jailbreak attack approach targeting LLMs," 2024, *arXiv:2409.07503*.

[30] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022, *arXiv:2211.09527*.

[31] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "YaRN: Efficient context window extension of large language models," 2023, *arXiv:2309.00071*.

[32] W. Xiong et al., "Effective long-context scaling of foundation models," 2023, *arXiv:2309.16039*.

[33] H. Tang et al., "RazorAttention: Efficient KV cache compression through retrieval heads," 2024, *arXiv:2407.15891*.

[34] J. Baek, A. Hussain, D. Liu, N. Vincent, and L. H. Kim, "Open WebUI: An open, extensible, and usable interface for AI interaction," 2025, *arXiv:2510.02546*.

[35] (2025). *Lobechat: An Open-Source, Modern Design CHATGPT/LLMS UI/framework*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/lobehub/lobe-chat

[36] Oobabooga. (2025). *Text Generation WEBUI: The Definitive Web UI For Local AI, With Powerful Features and Easy Setup* Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/oobabooga/text-generation-webui

[37] (2025). *Streamlit: A Faster Way To Build and Share Data Apps*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/streamlit/streamlit

[38] (2025). *Gradio: Build and Share Delightful Machine Learning Apps, All in Python*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/gradio-app/gradio

[39] (2025). *Jan is an Open Source Alternative To Chatgpt That Runs 100% Offline on Your Computer*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/janhq/jan

[40] (2025). *Chatbox: User-Friendly Desktop Client App for Ai Models/llms*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/chatboxai/chatbox

[41] Danny-avila. (2025). *Librechat*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/danny-avila/LibreChat

[42] SillyTavern. (2025). *Sillytavern: Llm Frontend for Power Users*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/SillyTavern/SillyTavern

[43] H. Face. (2025). *Chat-UI: Open Source Codebase Powering the Huggingchat App*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/huggingface/chat-ui

[44] (2025). *Bionic-GPT: Bionic is an On-Premise Replacement for Chatgpt, Offering the Advantages of Generative Ai While Maintaining Strict Data Confidentiality*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/bionic-gpt/bionic-gpt

[45] X. Qi et al., "Safety alignment should be made more than just a few tokens deep," 2024, *arXiv:2406.05946*.

[46] N. Rajani, L. Tunstall, E. Beeching, N. Lambert, A. M. Rush, and T. Wolf. (2023). *No Robots*. [Online]. Available: https://huggingface.co/datasets/HuggingFaceH4/norobots

[47] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, "Length-controlled AlpacaEval: A simple way to debias automatic evaluators," 2024, *arXiv:2404.04475*.

[48] Hugging Face. (2025). *Open LLM Leaderboard*. Accessed: Nov. 8, 2025. [Online]. Available: https://huggingface.co/spaces/open-llm-leaderboard/openllmleaderboard

[49] Z. Zhang et al., "H2O: heavy-hitter Oracle for efficient generative inference of large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 34661–34710.

[50] P. Chao et al., "JailbreakBench: An open robustness benchmark for jailbreaking large language models," 2024, *arXiv:2404.01318*.

[51] S. Bird, E. Klein, and E. Loper, *Natural Language Processing With Python: Analyzing Text With the Natural Language Toolkit*. Sebastopol, CA, USA: O'Reilly, 2009.

[52] B. Cao, Y. Cao, L. Lin, and J. Chen, "Defending against alignment-breaking attacks via robustly aligned LLM," 2023, *arXiv:2309.14348*.

[53] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[54] X. Qi et al., "Fine-tuning aligned language models compromises safety, even when users do not intend to!," 2023, *arXiv:2310.03693*.

[55] P. Ding et al., "A wolf in Sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily," 2023, *arXiv:2311.08268*.

[56] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Apr. 2023, pp. 32–42.

[57] C. Xiong, X. Qi, P.-Y. Chen, and T.-Y. Ho, "Defensive prompt patch: A robust and interpretable defense of LLMs against jailbreak attacks," 2024, *arXiv:2405.20099*.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

[58] G. Deng et al., "MASTERKEY: Automated jailbreaking of large language model chatbots," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–16.
[59] Z. Chang, M. Li, Y. Liu, J. Wang, Q. Wang, and Y. Liu, "Play guessing game with LLM: Indirect jailbreak attack with implicit clues," 2024, *arXiv:2402.09091*.
[60] Y. Zeng, Y. Wu, X. Zhang, H. Wang, and Q. Wu, "AutoDefense: Multi-agent LLM defense against jailbreak attacks," 2024, *arXiv:2403.04783*.
[61] L. Team. (2024). *Meta Llama Guard 2*. [Online]. Available: https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard2/MODELCARD.md
[62] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
[63] Y. Zhang, L. Ding, L. Zhang, and D. Tao, "Intention analysis makes LLMs a good jailbreak defender," in *Proc. 31st Int. Conf. Comput. Linguistics*, 2024, pp. 2947–2968.
[64] C. Barrett et al., "Identifying and mitigating the security risks of generative AI," *Found. Trends Privacy Secur.*, vol. 6, no. 1, pp. 1–52, 2023.
[65] Y. Fu, Y. Li, W. Xiao, C. Liu, and Y. Dong, "Safety alignment in NLP tasks: Weakly aligned summarization as an in-context attack," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, Aug. 2024, pp. 8483–8502. [Online]. Available: https://aclanthology.org/2024.acl-long.461
[66] J. Ji et al., "BeaverTails: Towards improved safety alignment of LLM via a human-preference dataset," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 24678–24704.
[67] AI@Meta. (2024). *Llama 3 Model Card*. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODELCARD.md
[68] (2024). *Openai Moderation Api*. [Online]. Available: https://platform.openai.com/docs/guides/moderation
[69] N. Jain et al., "Baseline defenses for adversarial attacks against aligned language models," 2023, *arXiv:2309.00614*.
[70] C. Zheng et al., "On prompt-driven safeguarding for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 61593–61613.
[71] M. Pisano et al., "Bergeron: Combating adversarial attacks through a conscience-based alignment framework," 2023, *arXiv:2312.00029*.
[72] Z. Xu, Y. Liu, G. Deng, Y. Li, and S. Picek, "A comprehensive study of jailbreak attack versus defense for large language models," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 7432–7449.
[73] S. Zhu et al., "AutoDAN: Interpretable gradient-based adversarial attacks on large language models," 2023, *arXiv:2310.15140*.
[74] J. Yu, X. Lin, Z. Yu, and X. Xing, "LLM-Fuzzer: Scaling assessment of large language model jailbreaks," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 4657–4674.
[75] S. Ge et al., "MART: Improving LLM safety with multi-round automatic red-teaming," 2023, *arXiv:2311.07689*.
[76] M. Phute et al., "LLM self defense: By self examination, LLMs know they are being tricked," 2023, *arXiv:2308.07308*.
[77] Y. Li, F. Wei, J. Zhao, C. Zhang, and H. Zhang, "RAIN: Your language models can align themselves without finetuning," 2023, *arXiv:2309.07124*.



Wenlong Meng received the B.S. degree in electronic information and communication from the Huazhong University of Science and Technology, Wuhan, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Zhejiang University, Hangzhou, China. His current research interests include trustworthy and privacy-preserving LLM applications. He has published papers on top-tier security conferences, such as USENIX Security, NDSS, and ACM CCS. Particularly, one of his research has been honored with the Distinguished Paper Award at NDSS 2024.



Fan Zhang is currently pursuing the bachelor's degree with Zhejiang University, Haining International Campus, China. He has been recommended for postgraduate admission to Zhejiang University and will pursue graduate studies there. His research interests include federated learning and the security of large language models (LLMs), with a focus on robust and privacy-preserving learning and trustworthy model deployment.



Wendao Yao is currently pursuing the dual B.Eng. degree in computer engineering with the ZJU-UIUC Institute, Zhejiang University, Haining, China, as part of a dual-degree program jointly offered by Zhejiang University and the University of Illinois Urbana-Champaign. His research interests include machine learning security, privacy-preserving natural language processing, and trustworthy large language models.



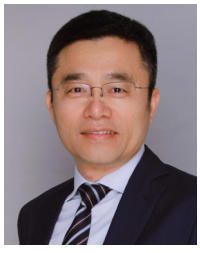
Zhenyuan Guo received the B.E. degree from Zhengzhou University and the M.E. degree from Beijing Institute of Technology. He is currently pursuing the Ph.D. degree with Zhejiang University. His research interests include large language models, long-context reasoning, and model security.



Yuwei Li received the Ph.D. degree in computer science and technology from Zhejiang University, Hangzhou, China, in 2021. She is an Associate Professor with the College of Electronic Engineering, National University of Defense Technology. Her research interests include system and software security, fuzz testing, and vulnerability detection.



Chengkun Wei received the Ph.D. degree in computer science from Zhejiang University. He is a ZJU 100-Young Professor with the College of Computer Science and Technology, Zhejiang University. He has published over ten papers in prestigious international journals and conferences, including CCS, NDSS, SIGCOMM, ATC, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, and IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY. His research interests include data security (differential privacy and AI security) and architecture design for cloud computing (network vSwitch and trusted execution environment (TEE) operating system design). He was honored with the NDSS 2024 Distinguished Paper Award. He is a reviewer of several international journals.



Wenzhi Chen (Member, IEEE) received the Ph.D. degree from the College of Computer Science and Technology, Zhejiang University. He is a Professor with the College of Computer Science and Technology, Zhejiang University; and the Director of the Information Technology Center, Zhejiang University. He is the Vice Dean of the College of Computer Science and Technology, Zhejiang University. His current research interests include embedded systems and their applications, computer architecture, computer system software, and information security. He is a member of ACM and the ACM Education Council. He was a recipient of the NDSS 2024 Distinguished Paper Award.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.